

Serving Static Files and Asset Organisation

Section: 9 — Views, Templates, and Static Assets

Subtopic: 02 — Static Files

Estimated Duration: 2–2.5 hours

Theory

Practical

Topics covered: `express.static` middleware, the `public` folder convention, serving CSS/JS/images, virtual path prefixes, asset organisation patterns, multiple static directories, cache headers (`ETag` , `Last-Modified` , `max-age`), cache-busting strategies, security considerations, integrating static assets with EJS templates

Learning Objectives — This Subtopic

- Configure `express.static` to serve CSS, JavaScript, images, and other files from a directory
- Organise static assets in a `public` folder following professional conventions
- Use virtual path prefixes to control how URLs map to the file system
- Understand how Express sets cache-related headers (`ETag` , `Last-Modified` , `max-age`) and configure them for production
- Implement cache-busting strategies to ensure browsers load updated assets
- Integrate static stylesheets and scripts into EJS templates

1. The Problem: How Do Browsers Get CSS and JavaScript?

When a browser loads an HTML page, it parses the markup and encounters references to external resources: `<link href="/css/style.css">` , `<script src="/js/app.js">` , `` . For each one, the browser sends a **separate HTTP request** back to the server.

Express routes handle dynamic requests — the response changes based on data, parameters, or logic. But CSS files, JavaScript files, images, fonts, and similar assets are **static**: they are the same for every request. Writing a route for every static file would be absurd:

anti-pattern.js

```
// Do NOT do this
const fs = require('fs');
const path = require('path');

app.get('/css/style.css', (req, res) => {
  const filePath = path.join(__dirname, 'public', 'css', 'style.css');
  res.setHeader('Content-Type', 'text/css');
  fs.createReadStream(filePath).pipe(res);
});

app.get('/js/app.js', (req, res) => {
  const filePath = path.join(__dirname, 'public', 'js', 'app.js');
  res.setHeader('Content-Type', 'application/javascript');
  fs.createReadStream(filePath).pipe(res);
});

// ... and so on for every single file
```

This is exactly what `express.static` replaces. One line of middleware serves an entire directory of files, with correct MIME types and caching headers handled automatically.

Analogy: Think of `express.static` as a self-service filing cabinet in a reception area. Instead of asking the receptionist (your route handlers) to fetch every document individually, visitors can open the cabinet and take what they need directly. The receptionist only handles requests that require judgement or personalisation.

2. Basic `express.static` Setup

Create a new project to work through the examples:

```
mkdir static-demo && cd static-demo
```

```
npm init -y
```

```
npm install express ejs
```

Stage 1 Serving a single static directory

```
static-demo/  
├─ node_modules/  
├─ public/  
│   ├─ css/  
│   │   └─ style.css  
│   ├─ js/  
│   │   └─ app.js  
│   └─ images/  
│       └─ logo.png  
├─ views/  
│   └─ home.ejs  
├─ app.js  
└─ package.json
```

Create the project files. First, a simple stylesheet:

```
public/css/style.css
```

```
body {  
  font-family: Arial, sans-serif;  
  max-width: 800px;  
  margin: 40px auto;  
  padding: 0 20px;  
  background-color: #f5f5f5;  
  color: #333;  
}  
  
h1 {  
  color: #0366d6;  
  border-bottom: 2px solid #0366d6;  
  padding-bottom: 8px;  
}  
  
.greeting {
```

```
background-color: #fff;
border: 1px solid #ddd;
border-radius: 6px;
padding: 16px;
margin-top: 16px;
}
```

A client-side JavaScript file:

public/js/app.js

```
document.addEventListener('DOMContentLoaded', () => {
  const greeting = document.getElementById('greeting');
  const hour = new Date().getHours();

  let message;
  if (hour < 12) {
    message = 'Good morning!';
  } else if (hour < 18) {
    message = 'Good afternoon!';
  } else {
    message = 'Good evening!';
  }

  greeting.textContent = message;
});
```

For the image, create a placeholder (or use any small PNG). A 1-pixel placeholder will do for testing:

```
mkdir -p public/images
```

On macOS/Linux:

```
echo "placeholder" > public/images/logo.png
```

On Windows:

```
echo placeholder > public\images\logo.png
```

Now create the EJS template that references these assets:

views/home.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Static Files Demo</title>
  <link rel="stylesheet" href="/css/style.css">
</head>
<body>
  <h1>Static Files Demo</h1>
  <div class="greeting">
    <p id="greeting">Loading...</p>
  </div>
  <p>Served at: <%= new Date().toLocaleTimeString() %></p>
  <script src="/js/app.js"></script>
</body>
</html>
```

And the Express application:

app.js

```
const express = require('express');
const path = require('path');

const app = express();

// — Serve static files from the 'public' directory —
app.use(express.static(path.join(__dirname, 'public')));

// — Template engine setup —
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/', (req, res) => {
  res.render('home');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

node app.js

Open `http://localhost:3000` in a browser. The page loads with styling from `style.css` and the greeting message is set by `app.js`.

Verify the static files are accessible directly:

```
curl -I http://localhost:3000/css/style.css
```

Output:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Thu, 27 Feb 2026 10:00:00 GMT
ETag: W/"1a3-18e5f5a0b00"
Content-Type: text/css; charset=UTF-8
Content-Length: 419
Date: Thu, 27 Feb 2026 10:05:00 GMT
Connection: keep-alive
```

What is happening?

- `app.use(express.static(path.join(__dirname, 'public')))` tells Express: "For any incoming request, first check if a matching file exists in the `public` directory. If it does, serve it. If not, move on to the next middleware or route."
- When the browser requests `/css/style.css`, Express maps this to `public/css/style.css` on disk. The `public` directory name itself does **not** appear in the URL.
- Express automatically sets the correct `Content-Type` header based on the file extension (`text/css` for `.css`, `application/javascript` for `.js`, `image/png` for `.png`, etc.).
- Express also sets cache-related headers: `ETag`, `Last-Modified`, and `Cache-Control`. More on these in Section 5.

Key Insight: The directory you pass to `express.static` is **not part of the URL**. The file at `public/css/style.css` is accessed via `/css/style.css`, not `/public/css/style.css`. The `public` folder is the root of the static file server.

3. Asset Organisation — The `public` Folder

The name `public` is a convention, not a requirement. You could call it `assets`, `static`, or `www`.

However, `public` is the standard in the Express ecosystem and it communicates intent clearly: everything in this folder is publicly accessible.

3.1 Recommended Folder Structure

For a typical Express application, organise static assets by type:

```
public/  
├─ css/  
│   ├── style.css  
│   ├── reset.css  
│   └── components/  
│       ├── navbar.css  
│       └── footer.css  
├─ js/  
│   ├── app.js  
│   ├── utils.js  
│   └── vendor/  
│       └── chart.min.js  
├─ images/  
│   ├── logo.png  
│   ├── favicon.ico  
│   └── icons/  
│       ├── search.svg  
│       └── menu.svg  
├─ fonts/  
│   ├── roboto-regular.woff2  
│   └── roboto-bold.woff2  
└─ downloads/  
    └── brochure.pdf
```

This structure has several advantages:

- **Predictable URLs** — developers and designers know that `/images/logo.png` corresponds to `public/images/logo.png`.

- **Clean separation** — stylesheets, scripts, images, and fonts are in their own directories.
- **Vendor isolation** — third-party files (e.g., `chart.min.js`) live in `vendor/` subdirectories, separate from your own code.
- **Easy deployment** — the entire `public` directory can be pushed to a CDN without any changes.

Important: Everything inside the `public` directory is accessible to anyone who knows (or guesses) the URL. Never place sensitive files here: no `.env` files, no configuration, no server-side code, no database credentials. Only assets intended for the browser belong in `public`.

4. Virtual Path Prefixes

By default, the URL path maps directly to the file path inside the static directory. You can add a **virtual prefix** — a path segment that exists in the URL but not on disk — by passing a mount path to `app.use()`.

Stage 2 Static files with a virtual prefix `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

// Files in 'public/' are served under the '/static' URL prefix
app.use('/static', express.static(path.join(__dirname, 'public')));

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/', (req, res) => {
  res.render('home-prefix');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

views/home-prefix.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Virtual Prefix Demo</title>
  <!-- Note: URL now starts with /static -->
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <h1>Virtual Prefix Demo</h1>
  <p>Stylesheet loaded from <code>/static/css/style.css</code></p>
  <script src="/static/js/app.js"></script>
</body>
</html>
```

node app.js

The stylesheet is now at `http://localhost:3000/static/css/style.css` instead of `http://localhost:3000/css/style.css`.

curl -I http://localhost:3000/static/css/style.css

Output:

```
HTTP/1.1 200 OK
Content-Type: text/css; charset=UTF-8
...
```

curl -I http://localhost:3000/css/style.css

Output:

```
HTTP/1.1 404 Not Found
...
```

What is happening?

- `app.use('/static', express.static(...))` means: "Only serve files from this directory when the URL starts with `/static`."

- A request for `/static/css/style.css` is resolved to `public/css/style.css` on disk. The `/static` prefix is stripped before looking up the file.
- The old URL `/css/style.css` (without the prefix) now returns 404.

Virtual prefixes are useful for several reasons:

- **Namespace clarity** — URLs like `/static/css/style.css` make it obvious at a glance that this is a static asset, not a dynamic route.
- **CDN migration** — If you later move assets to a CDN, replacing `/static` with `https://cdn.example.com` in your templates is straightforward.
- **Avoiding route conflicts** — If you have a dynamic route `/images/:id`, a virtual prefix like `/assets/images/logo.png` avoids ambiguity.

5. Multiple Static Directories

You can call `express.static` multiple times. Express checks each directory in the order they are registered and serves the first match.

Stage 3 Serving from multiple directories

```
static-demo/
├─ public/
│   └─ css/
│       └─ style.css
├─ uploads/
│   └─ avatar-001.png
├─ app.js
└─ ...
```

app.js

```
const express = require('express');
const path = require('path');
```

```
const app = express();

// Check 'public' first
app.use(express.static(path.join(__dirname, 'public')));

// Then check 'uploads'
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

app.get('/', (req, res) => {
  res.send('');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

```
node app.js
```

```
curl -I http://localhost:3000/css/style.css
```

Output:

```
HTTP/1.1 200 OK
Content-Type: text/css; charset=UTF-8
...
```

```
curl -I http://localhost:3000/uploads/avatar-001.png
```

Output:

```
HTTP/1.1 200 OK
Content-Type: image/png
...
```

What is happening?

- The first `express.static` handles requests for any file in `public/`.
- The second one serves files from the `uploads/` directory, but only when the URL starts with `/uploads`.

- This pattern is common when user-uploaded files are stored separately from application assets. The `uploads` directory often lives outside `public` and may have different permissions or backup strategies.

Key Insight: If two static directories contain a file at the same path, the one registered **first** wins. Express stops looking after the first match. Order matters.

6. Cache Headers — How Browsers Avoid Re-downloading

When a browser requests a static file, it stores the response in its local cache. On subsequent requests, it can either use the cached version or ask the server if the file has changed. This is controlled by HTTP cache headers. `express.static` sets three key headers automatically.

6.1 ETag

An ETag (Entity Tag) is a fingerprint of the file. Express generates it from the file's size and last-modified timestamp. When the browser makes a subsequent request, it sends the stored ETag in an `If-None-Match` header. If the file has not changed, the server responds with `304 Not Modified` (no body), and the browser uses its cached copy.

Stage 4 Observing ETag and 304 responses

Using the Stage 1 app, make a first request:

```
curl -v http://localhost:3000/css/style.css 2>&1 | grep -E "ETag|HTTP|Content-Length"
```

Output:

```
< HTTP/1.1 200 OK
< ETag: W/"1a3-18e5f5a0b00"
< Content-Length: 419
```

Now make a second request, sending the ETag back:

```
curl -v -H 'If-None-Match: W/"1a3-18e5f5a0b00"' http://localhost:3000/css/style.css 2>&1 | grep -E "HTTP|Content-Length"
```

Output:

```
< HTTP/1.1 304 Not Modified
```

The server responded with `304` and sent **no body**. The browser saves bandwidth by using the version it already has.

6.2 Last-Modified

`Last-Modified` works similarly to `ETag` but uses the file's modification timestamp. The browser sends `If-Modified-Since` on subsequent requests. If the file has not changed since that date, the server responds with `304`.

```
curl -I http://localhost:3000/css/style.css 2>&1 | grep Last-Modified
```

Output:

```
Last-Modified: Thu, 27 Feb 2026 10:00:00 GMT
```

6.3 Cache-Control and max-age

The `Cache-Control` header tells the browser *how long* it can use the cached file without even asking the server. By default, `express.static` sets `Cache-Control: public, max-age=0`, which means: "Cache the file, but always revalidate with the server before using it."

For production, you typically want a longer `max-age` so browsers do not need to contact the server at all for frequently used assets.

Stage 5 Configuring `max-age` `app.js`

```
const express = require('express');
const path = require('path');

const app = express();
```

```
// Cache static files for 1 day (86400 seconds) in production
const staticOptions = {
  maxAge: process.env.NODE_ENV === 'production' ? '1d' : 0
};

app.use(express.static(path.join(__dirname, 'public'), staticOptions));

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/', (req, res) => {
  res.render('home');
});

app.listen(3000, () => {
  console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode`);
  console.log('http://localhost:3000');
});
```

Test in development (default):

```
node app.js
```

```
curl -I http://localhost:3000/css/style.css 2>&1 | grep Cache-Control
```

Output:

```
Cache-Control: public, max-age=0
```

Test in production mode:

On macOS/Linux:

```
NODE_ENV=production node app.js
```

On Windows (cmd):

```
set NODE_ENV=production && node app.js
```

On Windows (PowerShell):

```
$env:NODE_ENV="production"; node app.js
```

```
curl -I http://localhost:3000/css/style.css 2>&1 | grep Cache-Control
```

Output:

```
Cache-Control: public, max-age=86400
```

What is happening?

- The `maxAge` option accepts milliseconds or a string like `'1d'` (1 day), `'1h'` (1 hour), or `'30m'` (30 minutes).
- With `max-age=86400`, the browser will serve the file from its local cache for up to 24 hours without contacting the server at all.
- In development, `max-age=0` ensures you always see the latest version of your assets when refreshing.

6.4 Summary of Cache Behaviour

Header	Set By	Purpose
<code>ETag</code>	<code>express.static</code> (automatic)	File fingerprint; enables 304 revalidation
<code>Last-Modified</code>	<code>express.static</code> (automatic)	File modification time; enables 304 revalidation
<code>Cache-Control: max-age</code>	<code>express.static</code> (configurable)	How long the browser can use the cached copy without revalidating

Key Insight: `ETag` and `Last-Modified` reduce bandwidth (304 has no body), but the browser still makes a request to the server. `max-age` eliminates the request entirely — the browser does not contact the server at all until the cache expires. For maximum performance, use `max-age` with cache-busting (covered next).

7. Cache-Busting Strategies

If you set a long `max-age` (e.g., one year), how do you force browsers to download a new version when the file changes? The answer is **cache-busting**: changing the URL so the browser treats it as a brand-new resource.

7.1 Query String Versioning

The simplest approach: append a version query parameter to the URL. When the file changes, change the version.

views/home.ejs

```
<link rel="stylesheet" href="/css/style.css?v=1.0.2">
<script src="/js/app.js?v=1.0.2"></script>
```

When you update `style.css`, change `?v=1.0.2` to `?v=1.0.3`. The browser sees a different URL and downloads the file fresh.

7.2 Automated Versioning with Package Version

Manual version bumping is error-prone. A common approach is to read the version from `package.json` and inject it into templates via `app.locals`.

Stage 6 Automated cache-busting app.js

```
const express = require('express');
const path = require('path');
const { version } = require('./package.json');

const app = express();

app.use(express.static(path.join(__dirname, 'public'), {
  maxAge: process.env.NODE_ENV === 'production' ? '7d' : 0
})));

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Make version available in all templates
```

```
app.locals.assetVersion = version;

app.get('/', (req, res) => {
  res.render('home-versioned');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

views/home-versioned.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Cache-Busting Demo</title>
  <link rel="stylesheet" href="/css/style.css?v=<%= assetVersion %>">
</head>
<body>
  <h1>Cache-Busting Demo</h1>
  <p>Assets served with version: <%= assetVersion %></p>
  <script src="/js/app.js?v=<%= assetVersion %>"></script>
</body>
</html>
```

`node app.js`

View the page source in the browser:

Output:

```
<link rel="stylesheet" href="/css/style.css?v=1.0.0">
...
<script src="/js/app.js?v=1.0.0"></script>
```

When you release a new version (`npm version patch` bumps to `1.0.1`), the URLs automatically change, busting the cache.

7.3 Filename Hashing (Advanced)

In larger projects, build tools (Webpack, Vite, esbuild) generate filenames with content hashes:

`style.a3f8b2.css` . When the content changes, the hash changes, and the URL changes automatically. This is the most robust strategy because it busts the cache per-file rather than globally. This is beyond the scope of this subtopic but worth knowing about for production workflows.

Warning: Some CDNs and proxies strip or ignore query strings when caching. If you deploy behind such infrastructure, filename hashing is more reliable than query string versioning.

8. All `express.static` Options

The options object passed to `express.static` supports several configuration properties beyond `maxAge` :

Option	Default	Description
<code>maxAge</code>	<code>0</code>	Sets the <code>Cache-Control: max-age</code> header (ms or string like <code>'1d'</code>)
<code>etag</code>	<code>true</code>	Enables or disables ETag generation
<code>lastModified</code>	<code>true</code>	Enables or disables the <code>Last-Modified</code> header
<code>dotfiles</code>	<code>'ignore'</code>	How to handle dotfiles (<code>.gitignore</code> , <code>.env</code>): <code>'allow'</code> , <code>'deny'</code> , <code>'ignore'</code>
<code>index</code>	<code>'index.html'</code>	The default file to serve when a directory is requested; set to <code>false</code> to disable

<code>extensions</code>	<code>false</code>	File extension fallbacks, e.g., <code>['html', 'htm']</code> so <code>/about</code> resolves to <code>about.html</code>
<code>redirect</code>	<code>true</code>	Redirect to trailing <code>/</code> when the path is a directory
<code>immutable</code>	<code>false</code>	Adds <code>immutable</code> directive to <code>Cache-Control</code> ; use with hashed filenames

Stage 7 Production-ready static configuration `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

const isProduction = process.env.NODE_ENV === 'production';

app.use(express.static(path.join(__dirname, 'public'), {
  maxAge: isProduction ? '7d' : 0,
  etag: true,
  lastModified: true,
  dotfiles: 'deny',           // Block access to .env, .gitignore, etc.
  index: false,               // Don't serve index.html for directory requests
  extensions: false           // Require exact file extensions in URLs
})));

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/', (req, res) => {
  res.render('home');
});

app.listen(3000, () => {
  console.log(`Server running in ${isProduction ? 'production' : 'development'} mode`);
  console.log('http://localhost:3000');
});
```

Test the `dotfiles: 'deny'` setting by creating a dotfile:

```
echo "SECRET=abc123" > public/.env
```

```
curl -I http://localhost:3000/.env
```

Output:

```
HTTP/1.1 403 Forbidden
```

Express returns `403` instead of serving the file. With the default `'ignore'` setting, it would return `404` (pretending the file does not exist). With `'deny'`, it explicitly rejects the request.

Key Insight: The `dotfiles: 'deny'` option is a safety net, but the real solution is to never place sensitive files in the `public` directory. Defence in depth: do not put them there, and also block access in case someone accidentally does.

9. Integrating Static Assets with EJS Templates

This section ties together everything from Subtopic 09-01 (Template Engines) and this subtopic. A realistic application has EJS templates that reference stylesheets, scripts, and images served by `express.static`.

Stage 8 Full integration — styled EJS pages with static assets

```
static-demo/  
├─ public/  
│  ├─ css/  
│  │   └─ style.css  
│  └─ js/  
│      └─ app.js  
├─ views/  
│  └─ home.ejs
```

```
|   └─ about.ejs  
|─ app.js  
└─ package.json
```

public/css/style.css

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
  
body {  
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;  
  background-color: #f5f7fa;  
  color: #333;  
  line-height: 1.6;  
}  
  
nav {  
  background-color: #24292e;  
  padding: 12px 20px;  
}  
  
nav a {  
  color: #fff;  
  text-decoration: none;  
  margin-right: 16px;  
  font-size: 0.95em;  
}  
  
nav a:hover {  
  text-decoration: underline;  
}  
  
.container {  
  max-width: 800px;  
  margin: 40px auto;  
  padding: 0 20px;  
}  
  
h1 {  
  color: #24292e;  
  margin-bottom: 16px;  
}
```

```
.card {
  background: #fff;
  border: 1px solid #e1e4e8;
  border-radius: 6px;
  padding: 20px;
  margin-bottom: 16px;
}

footer {
  margin-top: 60px;
  padding: 20px;
  text-align: centre;
  color: #6a737d;
  font-size: 0.85em;
  border-top: 1px solid #e1e4e8;
}
```

public/js/app.js

```
document.addEventListener('DOMContentLoaded', () => {
  // Highlight the active nav link
  const links = document.querySelectorAll('nav a');
  const currentPath = window.location.pathname;

  links.forEach(link => {
    if (link.getAttribute('href') === currentPath) {
      link.style.fontWeight = 'bold';
      link.style.textDecoration = 'underline';
    }
  });

  console.log('Client-side JS loaded successfully.');
```

app.js

```
const express = require('express');
const path = require('path');
const { version } = require('./package.json');

const app = express();

const isProduction = process.env.NODE_ENV === 'production';
```

```
// Static files
app.use(express.static(path.join(__dirname, 'public'), {
  maxAge: isProduction ? '7d' : 0,
  dotfiles: 'deny'
}));

// Template engine
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Global template variables
app.locals.appName = 'StaticDemo';
app.locals.currentYear = new Date().getFullYear();
app.locals.assetVersion = version;

// Routes
app.get('/', (req, res) => {
  res.render('home', {
    title: 'Home',
    features: [
      'Automatic MIME type detection',
      'ETag and Last-Modified headers',
      'Configurable cache control',
      'Virtual path prefixes',
      'Dotfile protection'
    ]
  });
});

app.get('/about', (req, res) => {
  res.render('about', {
    title: 'About'
  });
});

app.listen(3000, () => {
  console.log(`Server running in ${isProduction ? 'production' : 'development'} mode`);
  console.log('http://localhost:3000');
});
```

views/home.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```

<meta charset="UTF-8">
<title><%= title %> - <%= appName %></title>
<link rel="stylesheet" href="/css/style.css?v=<%= assetVersion %>">
</head>
<body>
  <nav>
    <a href="/"><%= appName %></a>
    <a href="/">Home</a>
    <a href="/about">About</a>
  </nav>

  <div class="container">
    <h1><%= title %></h1>

    <div class="card">
      <h3>express.static Features</h3>
      <ul>
        <% features.forEach(feature => { %>
          <li><%= feature %></li>
        <% }); %>
      </ul>
    </div>

    <footer>&copy; <%= currentYear %> <%= appName %></footer>
  </div>

  <script src="/js/app.js?v=<%= assetVersion %>"></script>
</body>
</html>

```

views/about.ejs

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title><%= title %> - <%= appName %></title>
  <link rel="stylesheet" href="/css/style.css?v=<%= assetVersion %>">
</head>
<body>
  <nav>
    <a href="/"><%= appName %></a>
    <a href="/">Home</a>
    <a href="/about">About</a>
  </nav>

```

```
<div class="container">
  <h1><%= title %></h1>

  <div class="card">
    <p>This is a demonstration of serving static files alongside EJS templates in
Express.</p>
    <p>The stylesheet, JavaScript, and any images are served by <code>express.static</
code> while the HTML is rendered dynamically by EJS.</p>
  </div>

  <footer>&copy; <%= currentYear %> <%= appName %></footer>
</div>

<script src="/js/app.js?v=<%= assetVersion %>"></script>
</body>
</html>
```

```
node app.js
```

Output:

```
// http://localhost:3000
```

```
StaticDemo Home About
```

```
Home
```

```
express.static Features
```

- Automatic MIME type detection
- ETag and Last-Modified headers
- Configurable cache control
- Virtual path prefixes
- Dotfile protection

```
© 2026 StaticDemo
```

Output:

```
// Browser console output:
```

```
Client-side JS loaded successfully.
```

This is the pattern you will use in production: `express.static` serves the CSS, JavaScript, images, and fonts. EJS renders the HTML. `app.locals` provides shared data (app name, year, asset version) to all templates.

Warning: Notice that both `home.ejs` and `about.ejs` duplicate the navigation, `<head>`, and footer. This duplication is addressed in Subtopic 09-03 (Layouts and Partials), where you extract shared markup into reusable `include()` files.

10. Middleware Order — Where to Place `express.static`

The position of `express.static` in your middleware stack matters. The recommended order is:

app.js (recommended middleware order)

```
const express = require('express');
const path = require('path');

const app = express();

// 1. Static files – checked first, fast path for assets
app.use(express.static(path.join(__dirname, 'public')));

// 2. Body parsers – only needed for dynamic routes
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// 3. Custom middleware (logging, auth, etc.)
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

// 4. Routes
app.get('/', (req, res) => {
  res.render('home');
});

// 5. 404 handler
app.use((req, res) => {
  res.status(404).send('Not Found');
});
```

Why place `express.static` first?

- Static file requests are the most common in a web application (every page load triggers multiple CSS/JS/image requests).
- Placing it before logging middleware means static requests do not clutter your logs with entries for `/css/style.css`, `/js/app.js`, etc.
- Placing it before body parsers avoids unnecessary overhead — static files do not have request bodies.
- If a static file matches, the middleware pipeline ends immediately. If no file matches, Express continues to the next middleware.

Key Insight: If you *do* want to log static file requests (e.g., for debugging or analytics), place your logging middleware *before* `express.static`. This is a deliberate trade-off between clean logs and visibility.

Summary

Concept	Key Point
<code>express.static(root)</code>	Built-in middleware that serves files from a directory; sets MIME types and cache headers automatically
<code>public</code> folder	Convention for the static root; everything inside is publicly accessible — never place sensitive files here
URL mapping	The static root directory is not part of the URL: <code>public/css/style.css</code> → <code>/css/style.css</code>
Virtual prefix	<code>app.use('/static', express.static(...))</code> adds a URL segment that does not exist on disk
Multiple directories	

	Call <code>express.static</code> multiple times; first match wins
<code>ETag</code> / <code>Last-Modified</code>	Enable 304 Not Modified responses, saving bandwidth but still requiring a server round-trip
<code>max-age</code>	Tells the browser to use the cached copy without contacting the server; set high in production
Cache-busting	Append a version query string (<code>?v=1.0.2</code>) or use filename hashing to force browsers to fetch updated files
<code>dotfiles: 'deny'</code>	Returns 403 for dotfiles (<code>.env</code> , <code>.gitignore</code>) as a security measure
Middleware order	Place <code>express.static</code> early in the stack for performance; before body parsers and custom middleware

Interview Questions

1. What does `express.static` do, and why is it preferable to writing individual routes for static files?

Expected answer: `express.static` is built-in middleware that serves files from a specified directory. It automatically handles MIME type detection, cache headers (ETag, Last-Modified, Cache-Control), and 304 responses. Writing individual routes would mean manually handling content types, streaming, caching, and error handling for every file — duplicating work that the middleware already does correctly.

2. If the `public` directory contains `public/images/logo.png`, what URL does the browser use to request it?

Expected answer: `/images/logo.png`. The directory name passed to `express.static` (in this case `public`) does not appear in the URL. It serves as the root, and the URL path maps to the relative path within that directory.

3. What is a virtual path prefix, and why would you use one?

Expected answer: A virtual path prefix is a URL segment added by `app.use('/prefix', express.static(...))` that does not correspond to a directory on disk. It makes static URLs more explicit (e.g., `/static/css/style.css`), avoids conflicts with dynamic routes, and simplifies future migration to a CDN by providing a single prefix to replace.

4. Explain the difference between `ETag / Last-Modified` and `max-age` in terms of caching behaviour.

Expected answer: `ETag` and `Last-Modified` enable **revalidation**: the browser contacts the server to check if the file has changed, and gets a 304 (no body) if it has not. This saves bandwidth but not the round-trip. `max-age` tells the browser to use the cached copy without contacting the server at all until the specified duration expires. `max-age` provides the best performance because it eliminates the request entirely.

5. You set `max-age` to one year for production. How do you ensure users get updated CSS after a deployment?

Expected answer: Use cache-busting: change the URL when the file changes. The simplest method is appending a version query string (e.g., `?v=1.0.3`). The most robust method is filename hashing with a build tool (e.g., `style.a3f8b2.css`). Since the URL changes, the browser treats it as a new resource and downloads it fresh.

6. Why should `express.static` typically be placed early in the middleware stack?

Expected answer: Static file requests are frequent (every page load triggers multiple asset requests). Placing `express.static` early means these requests are handled immediately without passing through unnecessary middleware like body parsers, authentication, or logging. If no matching file is found, the request passes to the next middleware as normal.

7. What does the `dotfiles: 'deny'` option do, and why is it important?

Expected answer: It causes `express.static` to return a `403 Forbidden` response for any request targeting a file that starts with a dot (e.g., `.env`, `.gitignore`). This prevents accidental exposure of sensitive configuration files. The default (`'ignore'`) returns `404`, which also hides the file but does not explicitly reject the request.

8. What security risk exists with the `public` directory, and how should it be mitigated?

Expected answer: Every file in the `public` directory is accessible to anyone via a direct URL. The risk is accidentally placing sensitive files (credentials, configuration, server code) there. Mitigation includes: never placing sensitive files in `public`, using `dotfiles: 'deny'`, reviewing the contents of `public` as part of code review, and adding `public` to version control with a clear understanding that it is a public-facing directory.

Practical Assignment: Portfolio Site with Static Assets

Objective: Build a multi-page Express application that serves static CSS, JavaScript, and images alongside dynamically rendered EJS pages, with proper cache configuration and asset versioning.

Setup

1. Create a new directory called `portfolio` and initialise:

```
mkdir portfolio && cd portfolio && npm init -y
```

2. Install dependencies:

```
npm install express ejs
```

Requirements

1. Create the following folder structure:

```
portfolio/  
├─ public/  
│   └─ css/
```

```
| | | style.css
| | | js/
| | | | main.js
| | | images/
| | | | (at least one image file)
| | views/
| | | home.ejs
| | | projects.ejs
| | | contact.ejs
| | app.js
| | package.json
```

2. Static middleware:

- Serve the `public` directory with `express.static`.
- Set `maxAge` to `'7d'` in production, `0` in development.
- Set `dotfiles` to `'deny'`.

3. Global template variables via `app.locals`:

- `siteName` — your chosen portfolio name
- `currentYear`
- `assetVersion` — read from `package.json`

4. Stylesheet (`public/css/style.css`): Style the site with a navigation bar, container layout, cards for project items, and a footer. Must be at least 40 lines of CSS.

5. Client-side JS (`public/js/main.js`): Add a small interactive feature — for example, highlight the active navigation link based on the current URL path, or toggle a "dark mode" class on `<body>`.

6. Three routes:

- `GET /` — Home page with a welcome message.
- `GET /projects` — Displays a list of at least 4 projects (name, description, technology tags). Pass the data from the route handler. Render with a loop.
- `GET /contact` — A static contact page (no form submission required).

7. Every template must reference the CSS and JS files using the `assetVersion` for cache-busting: `/css/style.css?v=<%= assetVersion %>`.

8. **Every template** must include a shared navigation bar and footer (duplicated for now — partials are covered in 09-03).

Test Cases

Verify with `curl` :

1. `curl -I http://localhost:3000/css/style.css` returns `200` with `Content-Type: text/css` and an `ETag` header.
2. `curl -I http://localhost:3000/.env` returns `403 Forbidden` (create a dummy `.env` in `public` to test).
3. Running with `NODE_ENV=production` , `curl -I http://localhost:3000/css/style.css` shows `max-age=604800` .
4. All three pages render correctly in a browser with styling and active navigation highlighting.

Expected Output — Projects Page

Visit `http://localhost:3000/projects` :

Output:

MyPortfolio Home Projects Contact

Projects

Task Manager

A full-stack task management app built with Express and EJS.

Tags: Node.js, Express, EJS

Weather Dashboard

Real-time weather data display using a public API.

Tags: JavaScript, API, CSS

Blog Engine

A server-rendered blog with markdown support.

Tags: Node.js, Express, Markdown

Chat Application

Real-time chat using WebSockets.

Tags: Node.js, Socket.io, JavaScript

© 2026 MyPortfolio

Expected Output — Cache Headers (Production)

```
NODE_ENV=production node app.js
```

```
curl -I http://localhost:3000/css/style.css
```

Output:

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=604800
ETag: W/"..."
Last-Modified: ...
Content-Type: text/css; charset=UTF-8
```

Bonus Challenge

Add a second static directory for user uploads: `app.use('/uploads', express.static(path.join(__dirname, 'uploads')))`. Create an `uploads` folder with a test image. On the home page, display this image using the `/uploads/...` URL. Set a shorter `maxAge` for uploads (e.g., `'1h'`) since user-uploaded content changes more frequently than application assets. Verify that `/uploads/test.png` returns a different `max-age` than `/css/style.css`.